

Course Code & Course Name: ITA05 & COMPUTER VISION

Assignment Title	A smart-city surveillance company is developing a real-time computer vision system for detecting objects through roadside cameras.
Course Outcome(s)- CO:	CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images. CO3: Analyse and extract image features using detection and matching techniques for effective image representation and comparison CO4: Compare object and pattern recognition techniques for interpreting visual data with spatial and motion characteristics.
Bloom's Taxonomy Level	BL4 – Analyze, BL5 – Evaluate
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities

Problem Statement: A smart-city surveillance company is developing a real-time system for detecting objects through roadside cameras. The deployed solution must have latency ≤ 100 ms, accuracy $\geq 85\%$, and operate using moderate computational resources (An edge device such as an **NVIDIA Jetson Nano / Jetson Xavier NX** (typical roadside-AI hardware)).

A city surveillance system must operate under the following conditions:

- If latency > 100 ms $\rightarrow$ system fails
- If accuracy $< 85\%$ $\rightarrow$ unacceptable
- Hardware supports only moderate computation

The engineering team is evaluating three models: Viola-Jones (78% accuracy, low latency), YOLO (92% accuracy, moderate latency), and Deep Learning model (95% accuracy, high latency). As the AI engineer, critically evaluate the suitability of each model against the operational constraints, determine the most appropriate model for deployment, and justify your decision using logical reasoning, performance trade-offs, and real-world surveillance requirements.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Computer Vision Fundamentals (CO1)	15	Clearly identifies the industrial defect-detection problem, accuracy requirement (>95%), product variations, and role of computer vision in automated inspection.	Identifies most requirements with minor gaps and demonstrates good understanding of the problem.	Basic understanding of defect detection is demonstrated, but some requirements are unclear.	Problem requirements are misunderstood or incorrectly interpreted.
Image Preprocessing, Enhancement & Feature Analysis (CO2)	20	Appropriately proposes preprocessing such as resizing, noise removal, illumination correction, filtering, segmentation, and relevant feature extraction for defect analysis.	Appropriate preprocessing and feature extraction techniques are selected with minor gaps.	Basic preprocessing techniques are identified but their application is limited.	Preprocessing/feature extraction techniques are inappropriate or missing.
Comparison of Traditional and Deep Learning Approaches (CO3/CO4)	20	Provides a comprehensive comparison based on accuracy, adaptability, computational cost, training requirements	Provides a relevant comparison covering most important factors.	Provides a basic comparison with limited analysis of performance and adaptability.	Comparison is incomplete, inaccurate, or lacks technical reasoning.

		, robustness, scalability, and deployment suitability.			
Solution Design & Model Selection (CO4/CO5)	15	Designs a suitable end-to-end defect detection pipeline and selects deep learning with strong justification for achieving >95% accuracy and adapting to product variations. Clearly addresses training, validation, and deployment.	Appropriate solution and model are selected with reasonable justification.	Solution is partially developed or model selection has limited justification.	No suitable solution design or model selection is provided.
Analysis, Evaluation & Justification (CO4)	10	Critically evaluates the performance –adaptability trade-off and convincingly justifies the selected approach using industrial requirements and practical deployment considerations.	Provides good analysis and justification with minor limitations.	Provides general analysis but limited critical evaluation.	Little or no meaningful analysis or justification.
Implementation, Results & Visualization (CO5)	10	Successfully implements the proposed approach using appropriate	Implementation works with minor issues and provides relevant results.	Partial implementation with limited or inconsistent results.	Implementation is missing, non-functional, or results are not demonstrated.

		computer vision tools; presents meaningful results, metrics, sample detections, and visualizations .			
Reflection, Industrial Relevance & SDG Mapping	10	Provides thoughtful reflection on design decisions, challenges, learning outcomes, industrial quality inspection, and clear SDG relevance.	Provides reasonable reflection and SDG connection.	Reflection is present but generic.	Reflection is missing or lacks meaningful connection to the problem.

1. Problem Understanding

A smart-city surveillance company is developing a real-time computer vision system to detect objects through roadside cameras. The system must run on an edge device such as an NVIDIA Jetson Nano or Jetson Xavier NX, where computing resources are limited. The system has three important requirements: latency must not exceed 100 ms, accuracy must be at least 85%, and the model must be able to run efficiently on moderate edge hardware without requiring a server-class GPU.

The engineering team is evaluating three models: Viola-Jones with 78% accuracy and low latency, YOLO with 92% accuracy and moderate latency, and a heavy Deep Learning model with 95% accuracy and high latency. As an AI engineer, critically evaluate these models based on accuracy, latency, and hardware requirements. Determine the most suitable model for deployment and justify your decision by considering constraint satisfaction, performance trade-offs, and real-world requirements of live surveillance systems. The decision should not be based only on accuracy, because a model that exceeds the 100 ms latency limit or requires excessive computational resources cannot be effectively used for real-time roadside surveillance.

2. Functional and Non-Functional Requirements

Requirement	Type	Description
Latency $\leq$ 100 ms	Non-functional (hard constraint)	End-to-end inference time per frame must not exceed 100 ms, or the system is considered failed for real-time roadside use.
Accuracy $\geq$ 85%	Non-functional (hard constraint)	Detection accuracy must be at least 85%, or the result is unacceptable for a safety-relevant surveillance application.
Moderate compute only	Non-functional (hardware constraint)	Must run on edge hardware such as an NVIDIA Jetson Nano / Xavier NX — no dedicated server-class GPU available at the roadside.
Object detection	Functional	Detect and localize relevant road objects (vehicles, pedestrians, signage) frame by frame from a live camera feed.
Continuous operation	Functional	Must sustain real-time throughput on a continuous video stream, not just single-image inference.

3. Image Preprocessing, Enhancement & Feature Analysis

Before detection, each frame from the roadside camera feed is processed as follows:

1. Resizing and Normalization – Each camera frame is resized to a fixed resolution to ensure consistent and predictable processing time.
2. Noise Removal – Gaussian or bilateral filtering is used to reduce rain effects, motion blur, and camera sensor noise.
3. Illumination Correction – Histogram equalization or CLAHE improves image visibility during day-night changes and reduces the effect of vehicle headlights.
4. Edge and Contrast Enhancement – Sobel or Canny operators are used to highlight important edges and structural features of vehicles and pedestrians.
5. Background Separation – Moving objects such as vehicles and pedestrians are separated from the static road background. This reduces the search area and can improve detection speed.
6. Feature Extraction – For classical models such as Viola-Jones, handcrafted features such as HOG can be used. YOLO and other deep learning models automatically learn important visual features using convolutional layers.

4. Comparison of Candidate Models

Each candidate is scored against the two hard numeric constraints (latency ≤ 100 ms, accuracy $\geq 85\%$) and the hardware's moderate-compute ceiling.

Model	Accuracy	Latency	Compute	Verdict
Viola-Jones	78%	Low	Very low	REJECTED – accuracy
YOLO (v8n)	92%	Moderate	Moderate	DEPLOYABLE
Deep Learning (2-stage)	95%	High	High	REJECTED – latency & compute

1 Robustness to Lighting, Angle, and Occlusion

- Viola-Jones: Poor performance under different lighting, viewing angles, and object occlusion.

- YOLO: Good performance in varying lighting conditions, angles, and partial occlusions.
- Deep Learning: Best robustness because of its ability to learn complex visual patterns.

2 Multi-Class Scalability

- Viola-Jones: Mainly suitable for specific/single-object detection and is less flexible for multiple classes.
- YOLO: Supports multi-class and multi-object detection, making it suitable for vehicles, pedestrians, traffic signs, etc.
- Deep Learning: Also supports multiple object classes and complex road scenes effectively.

3 Edge-Deployment Suitability

- Viola-Jones: Requires low computation but does not provide the accuracy needed for this application.
- YOLO: Highly suitable for real-time edge deployment and can be optimized using technologies such as TensorRT.
- Deep Learning: Requires more powerful GPU resources and may need pruning or quantization to run efficiently on moderate edge devices.

4 Overall Suitability

- Viola-Jones: Fast but fails the 85% accuracy requirement.
- YOLO: Provides a good balance of accuracy, speed, and computational efficiency.
- Deep Learning: Highly accurate but may fail the 100 ms latency and hardware constraints.

5. Selected Approach and Justification

Selected model: YOLO (a lightweight, edge-optimized variant such as YOLOv8n / YOLO-tiny).

Constraint-based reasoning:

1. Viola-Jones — accuracy = 78% < 85% required → REJECTED outright, regardless of its excellent latency. Speed cannot compensate for an unacceptable detection rate in a safety-relevant application.
2. Deep Learning (2-stage) — accuracy = 95% (highest of the three) but latency is high and, on moderate edge compute, is expected to exceed the 100 ms real-time budget → REJECTED. A highly accurate model that cannot keep pace with the live camera frame rate is operationally useless for continuous surveillance.
3. YOLO — accuracy = 92% ≥ 85% AND latency is moderate, and can be tuned to remain ≤ 100 ms on Jetson-class hardware (e.g., with TensorRT INT8 optimization) → ACCEPTED. It is the only candidate that satisfies both hard constraints simultaneously while matching the compute profile of the target hardware.

Real-world justification: roadside surveillance needs sustained, real-time throughput far more than it needs peak offline accuracy. A slightly-less-accurate model that never misses its time budget is more valuable operationally than a highly accurate model that periodically stalls or drops frames — which, in a traffic-safety context, translates directly into missed detections in real time regardless of the model's benchmark accuracy.

6. Algorithm / Pseudocode

6.1 Model Selection Logic

```
BEGIN SelectModel(candidates, LATENCY_LIMIT = 100 ms, ACCURACY_LIMIT = 85%)
```

```
  FOR EACH model IN candidates:
```

```
    reasons <- empty list
```

```
    ok <- TRUE
```

```
    IF model.latency > LATENCY_LIMIT:
```

```
      ok <- FALSE
```

```
      reasons.append("latency exceeds limit -> system fails")
```

```
    IF model.accuracy < ACCURACY_LIMIT:
```

```
      ok <- FALSE
```

```
      reasons.append("accuracy below limit -> unacceptable")
```

```
    IF model.compute == "high":
```

```
      ok <- FALSE
```

```
      reasons.append("exceeds moderate edge-device budget")
```



```

model.verdict <- ok ? "DEPLOYABLE" : "REJECTED"
model.reasons <- reasons

```

```

deployable <- FILTER candidates WHERE verdict == "DEPLOYABLE"
RETURN deployable[0] IF deployable is not empty ELSE "No model qualifies"
END

```

6.2 End-to-End Detection Pipeline

```

BEGIN DetectionPipeline(frame)
  frame <- Resize(frame, fixed_resolution)
  frame <- DenoiseFilter(frame)           // Gaussian / bilateral
  frame <- IlluminationCorrect(frame)     // CLAHE / histogram eq.
  roi <- BackgroundSubtract(frame)       // isolate moving objects

  t_start <- CurrentTime()
  detections <- YOLO_Infer(roi)           // forward pass
  detections <- NonMaxSuppression(detections)
  t_latency <- CurrentTime() - t_start

  IF t_latency > LATENCY_LIMIT:
    LogFrameDropped()
  ELSE:
    DrawBoundingBoxes(frame, detections)
    EmitToDashboard(frame, detections, t_latency)

  RETURN detections, t_latency
END

```

7. Reflection and SDG Relevance

- SDG 9 — Industry, Innovation and Infrastructure: demonstrates efficient edge-AI deployment that does not depend on constant cloud connectivity, supporting resilient, innovative city infrastructure.
- SDG 11 — Sustainable Cities and Communities: real-time roadside object detection directly supports traffic safety, pedestrian protection, and congestion management — core needs of sustainable urban mobility.

The key engineering takeaway from this exercise is that model selection under real-world deployment constraints is a trade-off problem, not an accuracy-maximization problem — the ability to reason about latency, compute, and accuracy jointly is what separates a lab-grade model from a deployable one.

SOURCE CODE:

"""

Flask backend for the Smart-City Roadside Object Detection dashboard.

Routes:

GET / -> dashboard UI

POST /api/detect -> run YOLO + Viola-Jones on an uploaded image,
return annotated images (as base64) + metrics

GET /api/compare -> run the fixed 3-model constraint comparison
(Viola-Jones / YOLO / Deep Learning) and
return the verdicts + chart

"""

```
import base64
```

```
import io
```

```
import time
```

```
from pathlib import Path
```

```
import cv2
```

```
import numpy as np
```

```
from flask import Flask, jsonify, render_template, request, send_from_directory
```

```
from ultralytics import YOLO
```

```
BASE_DIR = Path(__file__).parent
```

```
UPLOAD_DIR = BASE_DIR / "uploads"
```

```
UPLOAD_DIR.mkdir(exist_ok=True)
```

```
app = Flask(__name__)
```

```
# Load YOLO once at startup (expensive to reload per-request)
```

```
yolo_model = YOLO("yolov8n.pt")
```

```
vj_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_fullbody.xml")
```

```
LATENCY_LIMIT_MS = 100
```

```
ACCURACY_LIMIT = 0.85
```

```
# Fixed, literature/assignment-given accuracy figures for the 3-model comparison
```

```
KNOWN_ACCURACY = {
```

```
    "Viola-Jones": 0.78,
```

```
    "YOLO (v8n)": 0.92,
```

```
    "Deep Learning (2-stage)": 0.95,
```

```
}
```

```
def image_to_base64(img_bgr):
```

```
    """Encode an OpenCV BGR image as a base64 PNG data URL."""
```

```
    ok, buf = cv2.imencode(".png", img_bgr)
```

```
    if not ok:
```

```
        return None
```

```
    b64 = base64.b64encode(buf.tobytes()).decode("utf-8")
```

```
    return f"data:image/png;base64,{b64}"
```

```
def run_viola_jones(img_bgr, n_runs=5):
```

```
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
```

```
    vj_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=4) # warm-up
```

```
    latencies = []
```

```
    boxes = []
```

```
    for _ in range(n_runs):
```

```
        start = time.perf_counter()
```

```
        boxes = vj_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=4)
```

```
latencies.append((time.perf_counter() - start) * 1000)
```

```
vis = img_bgr.copy()
```

```
for (x, y, w, h) in boxes:
```

```
    cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 0, 255), 2)
```

```
return {
```

```
    "latency_ms": round(float(np.mean(latencies)), 2),
```

```
    "detections": int(len(boxes)),
```

```
    "image": image_to_base64(vis),
```

```
}
```

```
def run_yolo(img_bgr, n_runs=3):
```

```
    yolo_model.predict(source=img_bgr, verbose=False) # warm-up
```

```
    latencies = []
```

```
    results = None
```

```
    for _ in range(n_runs):
```

```
        start = time.perf_counter()
```

```
        results = yolo_model.predict(source=img_bgr, verbose=False)
```

```
        latencies.append((time.perf_counter() - start) * 1000)
```

```
    r = results[0]
```

```
    confs = r.bboxes.conf.cpu().numpy() if r.bboxes is not None else np.array([])
```

```
    annotated = r.plot()
```

```
    return {
```

```
        "latency_ms": round(float(np.mean(latencies)), 2),
```

```

    "detections": int(len(confs)),
    "avg_confidence": round(float(np.mean(confs)), 3) if len(confs) else 0.0,
    "image": image_to_base64(annotated),
}

```

```

def check_constraints(name, accuracy, latency_ms, compute):

```

```

    reasons = []

```

```

    ok = True

```

```

    if latency_ms > LATENCY_LIMIT_MS:

```

```

        ok = False

```

```

        reasons.append(f'latency {latency_ms:.1f} ms > {LATENCY_LIMIT_MS} ms limit \u2192
system fails')

```

```

    if accuracy < ACCURACY_LIMIT:

```

```

        ok = False

```

```

        reasons.append(f'accuracy {accuracy*100:.1f}% < {ACCURACY_LIMIT*100:.0f}% required
\u2192 unacceptable')

```

```

    if compute == "high":

```

```

        ok = False

```

```

        reasons.append("requires high compute \u2192 exceeds moderate edge-device budget")

```

```

    return {

```

```

        "model": name,

```

```

        "accuracy_pct": round(accuracy * 100, 1),

```

```

        "latency_ms": round(latency_ms, 2),

```

```

        "compute": compute,

```

```

        "verdict": "DEPLOYABLE" if ok else "REJECTED",

```

```

        "reasons": reasons or ["meets all constraints"],

```

```

    }

```

```

@app.route("/")

```

```

def index():
    return render_template("index.html")

@app.route("/images/<path:filename>")
def serve_sample_image(filename):
    return send_from_directory(BASE_DIR / "images", filename)

@app.route("/api/detect", methods=["POST"])
def api_detect():
    if "image" not in request.files:
        return jsonify({"error": "No image uploaded"}), 400

    file = request.files["image"]
    file_bytes = np.frombuffer(file.read(), np.uint8)
    img_bgr = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)

    if img_bgr is None:
        return jsonify({"error": "Could not decode image"}), 400

    vj_result = run_viola_jones(img_bgr)
    yolo_result = run_yolo(img_bgr)

    vj_verdict = check_constraints("Viola-Jones", KNOWN_ACCURACY["Viola-Jones"],
    vj_result["latency_ms"], "low")

    yolo_verdict = check_constraints("YOLO (v8n)", KNOWN_ACCURACY["YOLO (v8n)"],
    yolo_result["latency_ms"], "moderate")

    return jsonify({
        "viola_jones": {**vj_result, **vj_verdict},
        "yolo": {**yolo_result, **yolo_verdict},

```

```
}}
```

```
@app.route("/api/compare")
```

```
def api_compare():
```

```
    """Runs the fixed 3-model comparison on a bundled sample image."""
```

```
    sample_path = BASE_DIR / "images" / "bus.jpg"
```

```
    img_bgr = cv2.imread(str(sample_path))
```

```
    vj_result = run_viola_jones(img_bgr)
```

```
    yolo_result = run_yolo(img_bgr)
```

```
    dl_latency = yolo_result["latency_ms"] * 5.0 # modeled: 2-stage ~5x slower than single-shot
```

```
    results = [
```

```
        check_constraints("Viola-Jones", KNOWN_ACCURACY["Viola-Jones"],  
vj_result["latency_ms"], "low"),
```

```
        check_constraints("YOLO (v8n)", KNOWN_ACCURACY["YOLO (v8n)"],  
yolo_result["latency_ms"], "moderate"),
```

```
        check_constraints("Deep Learning (2-stage)", KNOWN_ACCURACY["Deep Learning (2-stage)"],  
dl_latency, "high"),
```

```
    ]
```

```
    deployable = [r for r in results if r["verdict"] == "DEPLOYABLE"]
```

```
    return jsonify({
```

```
        "constraints": {"latency_limit_ms": LATENCY_LIMIT_MS, "accuracy_limit_pct":  
ACCURACY_LIMIT * 100},
```

```
        "results": results,
```

```
        "selected_model": deployable[0]["model"] if deployable else None,
```

```
        "sample_images": {
```

```

        "viola_jones": vj_result["image"],

        "yolo": yolo_result["image"],

    },

})

if __name__ == "__main__":

    app.run(debug=True, host="0.0.0.0", port=5000)

```

OUTPUT:

127.0.0.1:5000

A smart-city surveillance feed only works if detection keeps up with the camera. This console runs real detectors against the operating limits — latency ≤ 100 ms, accuracy $\geq 85\%$, moderate edge compute — and shows, frame by frame, which one actually qualifies.

01 — Test a frame

Upload a roadside/traffic image. It runs through both detectors live.

Use bundled sample instead

Run detection

Done.

Viola-Jones

REJECTED

Metric	Value
LATENCY	2119.8 ms
ACCURACY	78.6%
DETECTIONS	4

latency 2119.8 ms > 100 ms limit → system fails - accuracy 78.6% < 85% required → unacceptable

YOLOv8n

DEPLOYABLE

Metric	Value
LATENCY	92.8 ms
ACCURACY	92.6%
DETECTIONS	38

meets all constraints

78.0% < 85% required → unacceptable

02 – Constraint check, all three candidates

Viola-Jones, YOLO, and a heavy two-stage detector, scored against the same operating limits.

Run comparison

Done.

MODEL	ACCURACY	LATENCY	COMPUTE	VERDICT
Viola-Jones	78%	1293.64 ms	low	REJECTED
YOLO (v8n)	92%	73.93 ms	moderate	DEPLOYABLE
Deep Learning (2-stage)	95%	369.65 ms	high	REJECTED

Selected for deployment: YOLO (v8n) – the only candidate meeting both the latency and accuracy limits on moderate edge compute.